



**Кафедра информационных систем
в химической технологии**

**Институт тонких химических технологий
им. М.В. Ломоносова
РТУ МИРЭА**



ИНФОРМАЦИОННЫЕ СИСТЕМЫ В ХТ

Лекция 15. Типы клиентов: веб-интерфейс

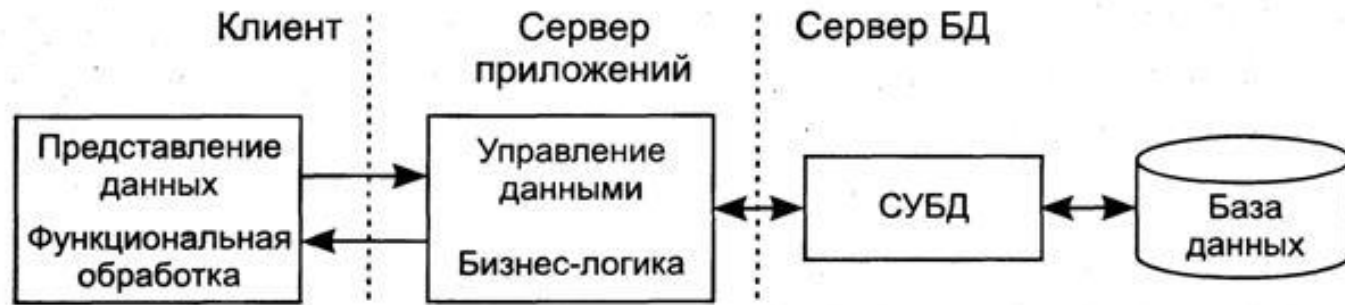
Содержание лекции

В данной лекции будут рассмотрены следующие темы:

-  Типы клиентских приложений
-  Веб-интерфейс
-  Каркасы для серверных приложений с веб-интерфейсом
-  Каркасы для веб-разработки

Клиент

- В клиент-серверной архитектуре часть приложения, ответственная за интерфейс пользователя



Типы клиентских приложений

Толстый клиент

- ⦿ Приложение, с графическим интерфейсом пользователя
- ⦿ Для подключения к серверу используются
 - Частные протоколы
 - Модель удаленного вызова (CORBA, RMI/IIOP и т.п.)
 - Модель веб-служб (SOAP, REST)

Клиент командной строки

- ⦿ Разновидность толстого клиента без GUI

Тонкий клиент

- ⦿ Графический интерфейс пользователя отображается в браузере
- ⦿ Для построения GUI применяются веб-технологии (HTML, CSS, JS)
- ⦿ Для подключения к серверу используется протокол HTTP

Клиент времени исполнения

- ⦿ Приложение, реализующее API для работы с сервером по сетевому подключению

Основные способы предоставления GUI (1/2)



Толстый клиент

- ⦿ Оконный интерфейс, опирающийся на возможности операционной системы
- ⦿ Достоинства:
 - "Богатый" интерфейс пользователя с неограниченным функционалом
 - Интеграция с возможностями ОС
 - Аутентификации и авторизация на стороне клиента
- ⦿ Недостатки:
 - Необходимость инсталляции (разный код для разных ОС)
 - Сложность обновления, при обновлении серверной части
 - Порты для частных протоколов могут быть закрыты правилами сети



Мобильно приложение

- ⦿ Частный случай толстого клиента
- ⦿ Стратегия Mobile First

Основные способы предоставления GUI (2/2)



Тонкий клиент

- ◉ Веб-интерфейс, опирающийся на возможности веб-браузера
- ◉ Достоинства:
 - Современные веб-интерфейсы по функционалу практически не отстают от GUI толстых клиентов
 - Не требует установки и обновления (нужен совместимый браузер)
 - Доступен на любом устройстве, где есть браузер (ПК, планшет, смартфон)
 - Протокол HTTP обычно пропускается межсетевыми экранами
- ◉ Недостатки:
 - Ограниченные возможности ("песочница" браузера)
 - Аутентификация только на стороне сервера

Эволюция веб-интерфейса (1/3)



"Тонкий" тонкий клиент

- ⦿ Браузер отображает статические HTML документы с минимумом встроенного кода
- ⦿ Каждое действие пользователя обрабатывается на сервере, как синхронный запрос нового документа
- ⦿ Документы динамически создаются на сервере
- ⦿ Достоинства:
 - Минимальные требования к поддержке веб-технологий в браузере
 - Минимальные требования к аппаратным ресурсам устройства
 - Код клиента размещен на сервере и защищен от просмотра пользователем или вмешательства в его выполнение
- ⦿ Недостатки:
 - Большой объем данных передается между клиентом и сервером
 - Высокие требования к скорости сетевого соединения
 - Большая нагрузка на сервер

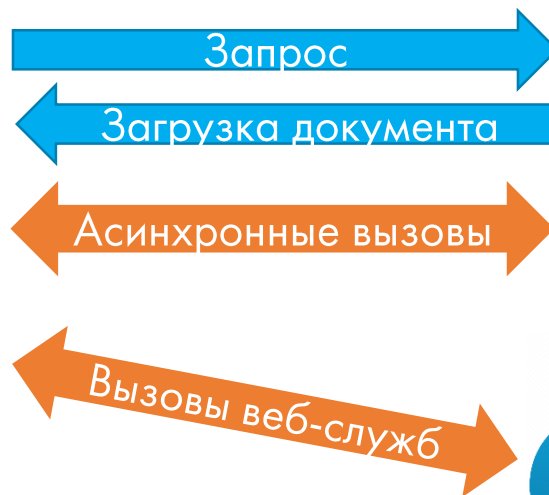
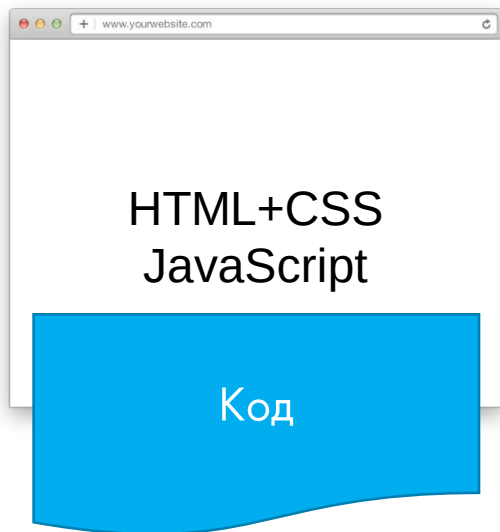
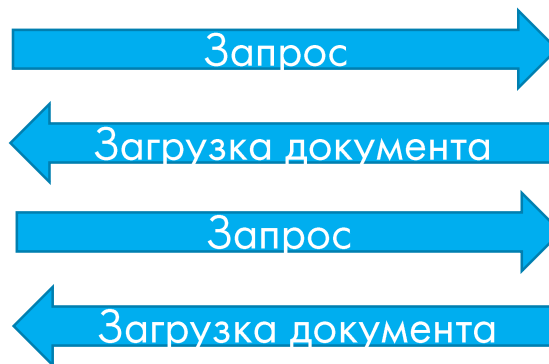
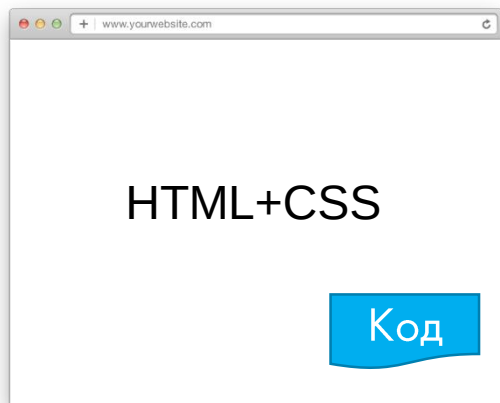
Эволюция веб-интерфейса (2/3)



"Толстый" тонкий клиент

- ⦿ Браузер отображает динамические HTML документы с большим объемом встроенного кода
- ⦿ Действия пользователя обрабатывается кодом, загруженным в браузер
- ⦿ Данные передаются на клиент через асинхронные вызовы (AJAX, API)
- ⦿ Достоинства:
 - Небольшое время отклика
 - Объем данных, передаваемых по сети, минимизирован
 - Сокращена нагрузка на сервер
 - "Богатый" интерфейс пользователя
- ⦿ Недостатки:
 - Рост требований к производительности клиентского устройства
 - Требуется поддержки современных веб-технологий в браузере
 - Код клиента загружен в браузер и доступен пользователю

Эволюция веб-интерфейса (3/3)



Популярные каркасы

Каркас	Язык	DI	Веб	ORM	Микросервисы	Cloud Native
Spring	Java	✓	✓	✓	✓	✓ (Boot)
Micronaut	Java	✓	✓	✓	✓	✓
Quarkus	Java	✓	✓	✓	✓	✓
NestJS	JS/TS	✓	✓	✗	✓	✓
Django	Python	✗	✓	✓	✗	✗
ASP.NET Core	C#	✓	✓	✓	✓	✓

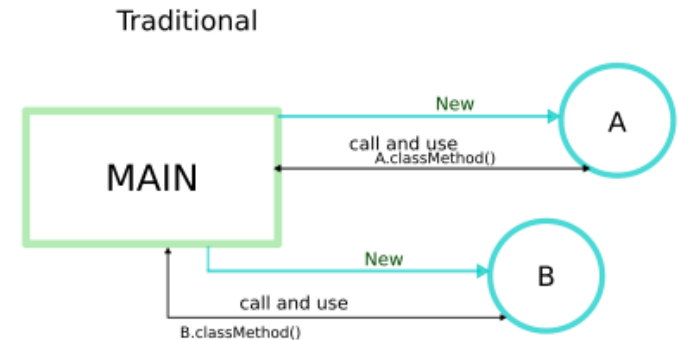
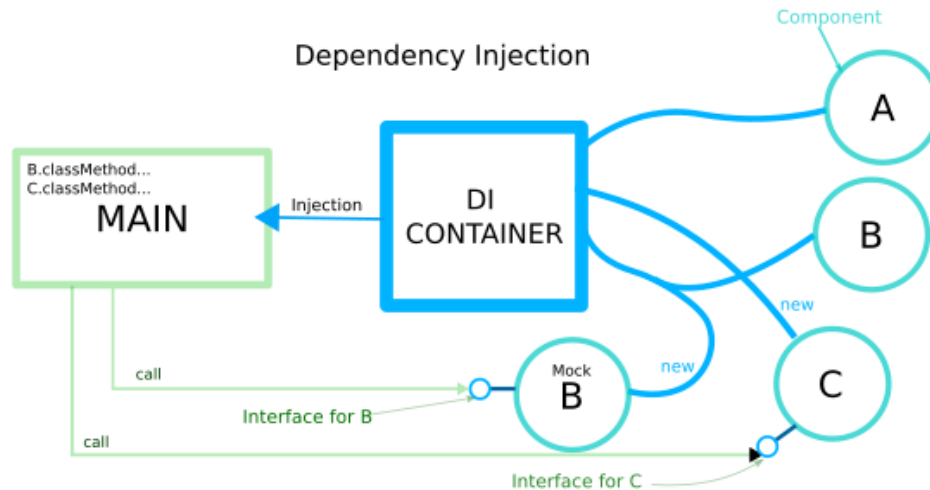
DI - Dependency Injection

- ⦿ *Внедрение зависимости* - предоставления внешней зависимости программному компоненту, форма *инверсии управления*

ORM - Object-Relational Mapping

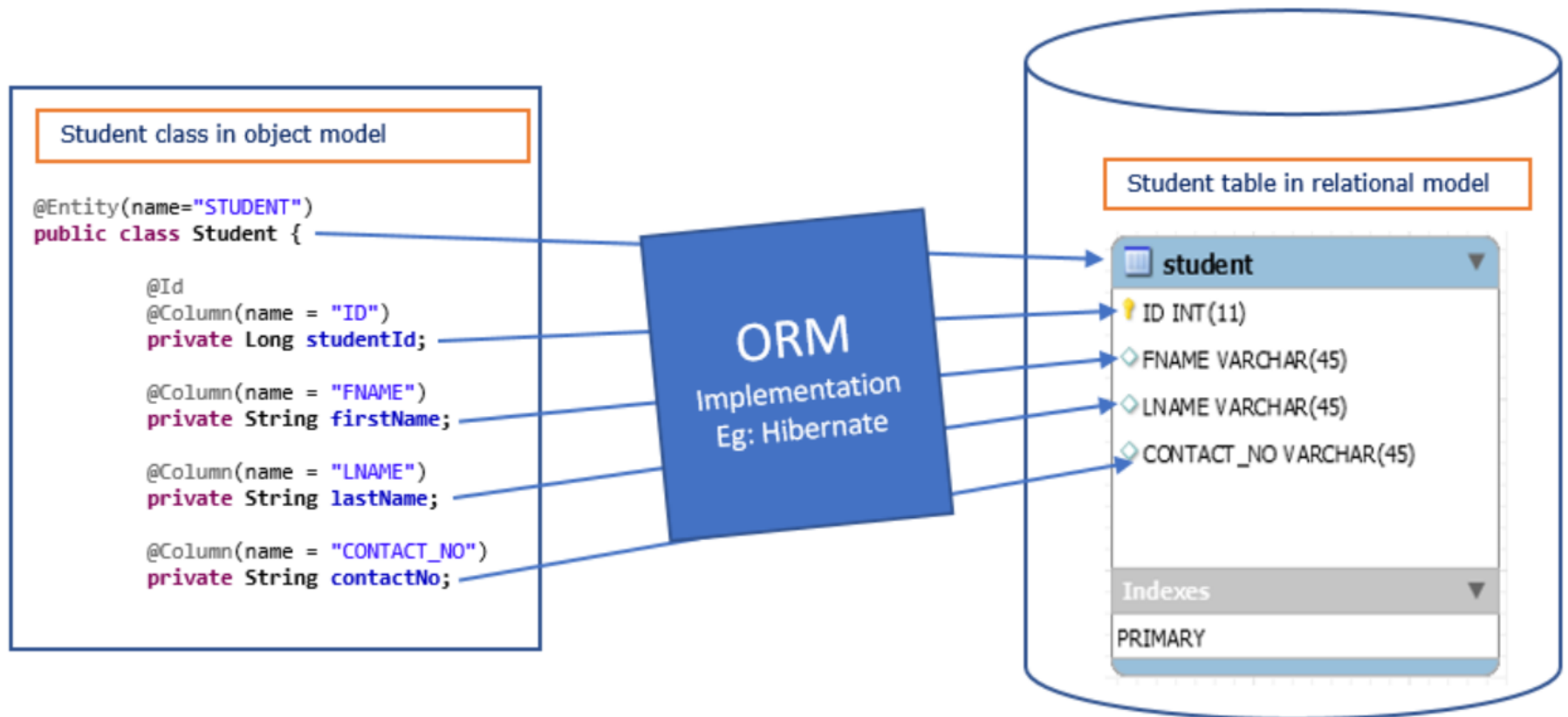
- ⦿ *Объектно-реляционное отображение* — технология программирования, которая связывает базы данных с концепциями объектно-ориентированных языков программирования

Внедрение зависимости



- 📡 **Инверсия управления** (*Inversion of Control, IoC*) — общий принцип программирования, особенно важный в рамках объектно-ориентированной парадигмы, используемый для уменьшения связанности. Архитектурное решение интеграции, упрощающее расширение возможностей системы, при котором поток управления программы контролируется каркасом.

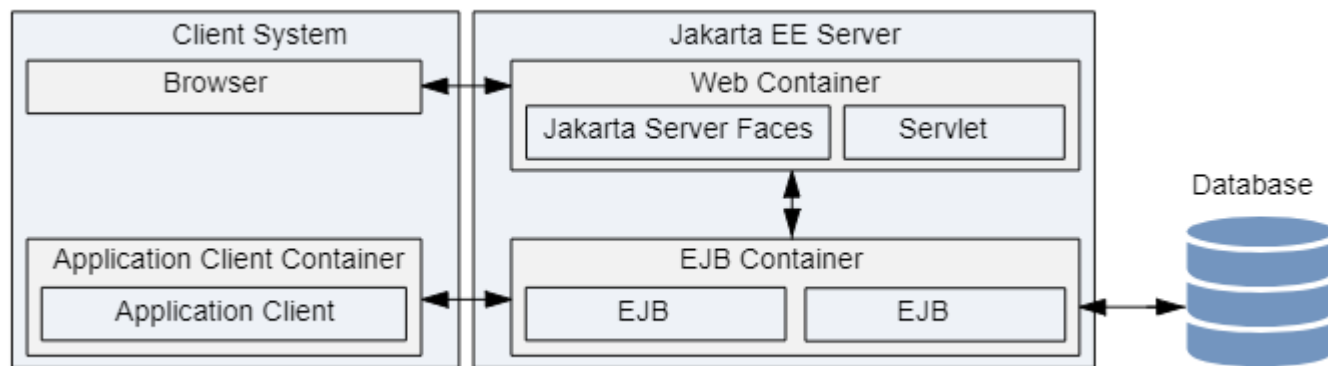
Объектно-реляционное отображение



Популярные Java каркасы (1/2)

Jakarta EE (ранее Java EE)

- Трехзвенная архитектура с сервером приложений (WildFly, OpenLiberty)
- Сегмент enterprise-решений



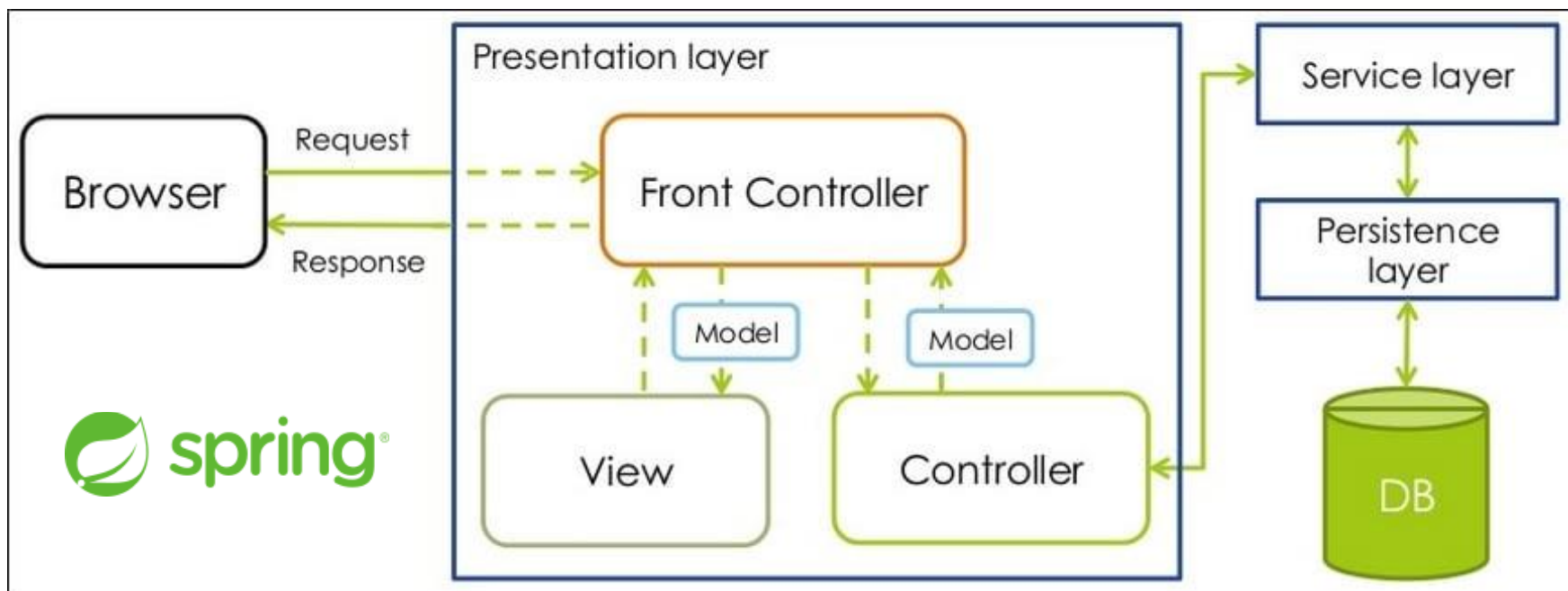
ApacheTomEE



Популярные Java каркасы (2/2)

🌐 Spring и Spring Boot

- ⦿ Самый востребованный Java каркас
- ⦿ Поддержка аспект-ориентированной парадигмы
- ⦿ Проще чем Jakarta EE, встроенный сервер приложений

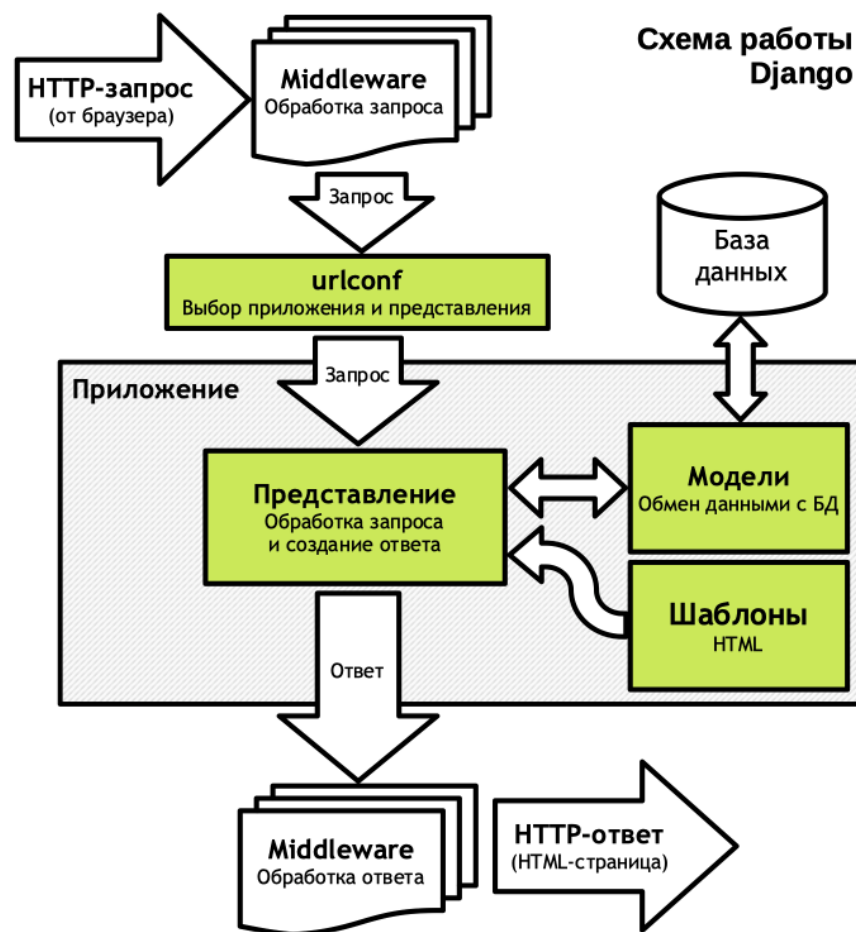


Популярные Python каркасы

Django / FastAPI

- Django — "Spring для Python":
- ORM
- Административный интерфейс
- Аутентификация "из коробки"
- Монолитный, но мощный
- FastAPI — легковесный аналог Spring Boot

django



Популярные каркасы

Ruby on Rails

- ⦿ Разработчик: David Heinemeier Hansson
- ⦿ Язык: Ruby
- ⦿ принцип «Convention over Configuration» (Соглашение вместо конфигурации), который кардинально меняет разработку веб-приложения, делая ее более интуитивной и продуктивной

ASP.NET Core

- ⦿ Разработчик: Microsoft
- ⦿ Язык: C#
- ⦿ Кроссплатформенный каркас, который используется для создания современных веб-приложений и API

Laravel

- ⦿ Разработчик: Taylor Otwell
- ⦿ Язык: PHP
- ⦿ Каркас для разработки веб-приложений, который следует шаблону MVC

Популярные back-end каркасы

Express.js

- ⦿ Разработчик: Strongloop и IBM
- ⦿ Язык: JavaScript
- ⦿ Самый популярный минималистичный веб-каркас на платформе Node.js, с помощью которого можно создавать гибкие HTTP-серверы с применением RESTful API

Flask

- ⦿ Разработчик: Armin Ronacher
- ⦿ Язык: Python
- ⦿ Крайне легковесный каркас, который идеально подходит для создания небольших и средних веб-приложений

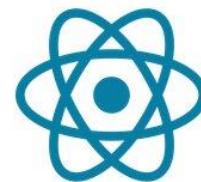
Phoenix

- ⦿ Разработчик: Phoenix Framework
- ⦿ Язык: Elixir
- ⦿ Современный веб-каркас для функционального языка программирования Elixir

Популярные front-end каркасы

React

- ⦿ Разработчик: Meta
- ⦿ Язык: JavaScript
- ⦿ Примеры: Facebook, Instagram, Airbnb, Netflix
- ⦿ Сегмент enterprise-решений



Angular

- ⦿ Разработчик: Google
- ⦿ Язык: TypeScript
- ⦿ Примеры: Google, Microsoft, PayPal



Vue

- ⦿ Разработчик: Эван Ю
- ⦿ Язык: JavaScript
- ⦿ Примеры: Alibaba, GitLab, Behance



Итоги

В данной лекции были рассмотрены следующие темы:

-  Типы клиентских приложений
-  Веб-интерфейс
-  Каркасы для серверных приложений с веб-интерфейсом
-  Каркасы для веб-разработки